
AME 323 - Gas Dynamics, Project 1: Solving the Taylor-Maccoll equations

Table of Contents

.....	1
Description:	1
Housekeeping:	1
Given Values:	1
Setting up the arrays and variables	2
Solving the System of Equations	2
Plotting	3
Solving the Taylor-Maccoll Equations	6
Taylor-Maccoll HELPER FUNCTIONS	7
Generic Helper Functions	9

Authors: Maren Kalberer, Etan Grant

Date: 3-27-26

Description:

Takes initial guess of beta (uses the wedge case), then uses ode45 to work backwards using the Taylor Maccoll equation to find a rough value of cone half-angle from that guess. Then, uses fzero() to repeat that process until there is zero error between the reverse-engineered cone half-angle and the input half-angle (they're the same now). Yay! Now that initial beta guess where fzero() found zero error is your conical oblique shock angle. Congratulations!

Housekeeping:

Requires having the a few toolboxes installed for certain functions

```
% Clear old variable values, outputs, and close old figures
close all;
clear;
clc;
```

Given Values:

Known relationships, and input parameters.

```
gamma = 1.4; % Ratio of specific heats for standard air

% The given mach values for which we need to generate curves:
M_inputs = [1.5, 2.0, 5.0];
```

Setting up the arrays and variables

Sets up system of θ , β , M , other crap.

```
% number of curves to plot (same as number of mach inputs)
num_curves = length(M_inputs);

% Range of theta_c (cone half-angle  $\theta$ ) input values to plot
theta_c_min = 0.5; % Left bound of the curve (degrees)
theta_c_max = 54; % Right bound of the curve (degrees)
num_points = 50;

theta_c = linspace(theta_c_min, theta_c_max, num_points); % 'x' coord values

% Set up an empty array for the output  $\beta$  values
% Each row: One M input
% Each column: The values of theta_c (see above definitions)
beta_outputs = zeros(num_curves, num_points);
beta_wedge_outputs = NaN(num_curves, num_points);

%max_betas = zeros(1, num_curves);
%max_thetas = max_betas;
```

Solving the System of Equations

Solves the system and logs the results. Uses other functions to do the math. Finds the cone shock angles given the above-defined M , θ inputs.

```
for i = 1:num_curves
    freestream_mach = M_inputs(i);

    [max_thetas, max_betas] = maxWedgeBeta(freestream_mach, gamma);

    % For each row (M value), run the beta, point-by point.
    % HORRIBLY inefficient, but we just need it to work first.
    for j = 1:num_points

        % Grab the input
        cone_angle = theta_c(j);

        % Store the Output
        beta_outputs(i,j) = coneBeta(freestream_mach, cone_angle, gamma);

        if cone_angle < max_thetas
            beta_wedge_outputs(i,j) = beta(freestream_mach, cone_angle,
gamma, 0);
        else
            beta_wedge_outputs(i,j) = NaN; % <-- Gap instead of zero
        end

    end

end
```

end

% Find the max beta and its angle

Plotting

This is where all plots are generated.

```
% UofA colors!
arizonaBlue = [12, 35, 75] ./ 255;
arizonaRed = [171, 5, 32] ./ 255;
midnight = [0, 28, 72] ./ 255;
azurite = [30, 82, 136] ./ 255;
oasis = [55, 141, 189] ./ 255;
chili = [139, 0, 21] ./ 255;
bloom = [239, 64, 86] ./ 255;
sky = [192, 211, 235] ./ 255;
leaf = [112, 184, 101] ./ 255;
river = [0, 125, 132] ./ 255;
mesa = [169, 92, 66] ./ 255;

% Set interpreter to Latex. Lets us use subscripts and greek characters.
set(groot, 'defaultTextInterpreter', 'latex');
set(groot, 'defaultAxesTickLabelInterpreter', 'latex');
set(groot, 'defaultLegendInterpreter', 'latex');

colors = {azurite, bloom, leaf}; % colors to cycle through

plot1 = false; % Toggle this plot on/off
if plot1
    fig1 = figure;
    ax1 = gca;

    % Text format
    lineWidth = 1.5;
    axisFontSize = 28;
    legendFontSize = 16;
    hold on;

    for i = 1:num_curves
        % Make a custom name "M = X.XX" for each curve and store for the
legend
        name = "$M = " + sprintf("%.2f", M_inputs(i)) + "$ (cone)";
        name2 = "$M = " + sprintf("%.2f", M_inputs(i)) + "$ (wedge)";

        % Grab the x ' $\theta$ ' and y ' $\beta$ ' values
        x = theta_c; % Row vector of cone angles
        y = beta_outputs(i,:); % Row vector of cone shock angles
        y_wedge = beta_wedge_outputs(i,:); % Same but wedge

        % Cycle through the colors (however many colors are defined)
        color = colors{mod(i-1, length(colors))+1};
```

```
% Plot the given curve, including the name (referenced in legend)
plot(x, y, "LineWidth", lineWidth, "Color", color, DisplayName=name);
plot(x, y_wedge, "LineWidth", lineWidth-0.5, "Color", color,
LineStyle = '--' , DisplayName=name2);
end

grid on;

% Put the legend and axis labels
legend(FontSize=legendFontSize,Location="southeast");
xlabel("Half-angle,  $\theta$ ", fontsize = axisFontSize);
ylabel("Oblique shock angle,  $\beta$ ", FontSize = axisFontSize);

ylim(ax1, [0, 75]);
xlim(ax1, [0, 55]);

% Sets axes at origin.
ax1.XAxisLocation = 'origin';
ax1.YAxisLocation = 'origin';

hold off; % Done with plot :)
end

% Plot for PDF (8.5x11 inch paper size & PDF export)
plot2 = true;
if plot2

    % 1. Create figure and force the "Paper" properties
    fig2 = figure('Units', 'inches', 'Position', [0, 0, 8.5, 11], 'Visible',
'on');

    % Force the PDF output size to be exactly 8.5x11
    set(fig2, 'PaperUnits', 'inches');
    set(fig2, 'PaperSize', [8.5 11]);

    % Sets the "Drawing Area" to be 7.5x10, centered (0.5" margins)
    % Format: [Left, Bottom, Width, Height]
    set(fig2, 'PaperPosition', [0.5 0.5 7.5 10]);
    set(fig2, 'PaperPositionMode', 'manual');

    ax2 = axes(fig2);
    hold(ax2, 'on');

    % Formatting
    lineWidth2 = 2.0;
    axisFontSize2 = 24;
    legendFontSize2 = 16;

    titlecolor = arizonaBlue;
    axistextcolor = arizonaBlue;

    for i = 1:num_curves
        name = "$M = " + sprintf("%.2f", M_inputs(i))+"$ (cone)";
        name2 = "$M = " + sprintf("%.2f", M_inputs(i))+"$ (wedge)";
```

```
color = colors{mod(i-1,length(colors))+1};

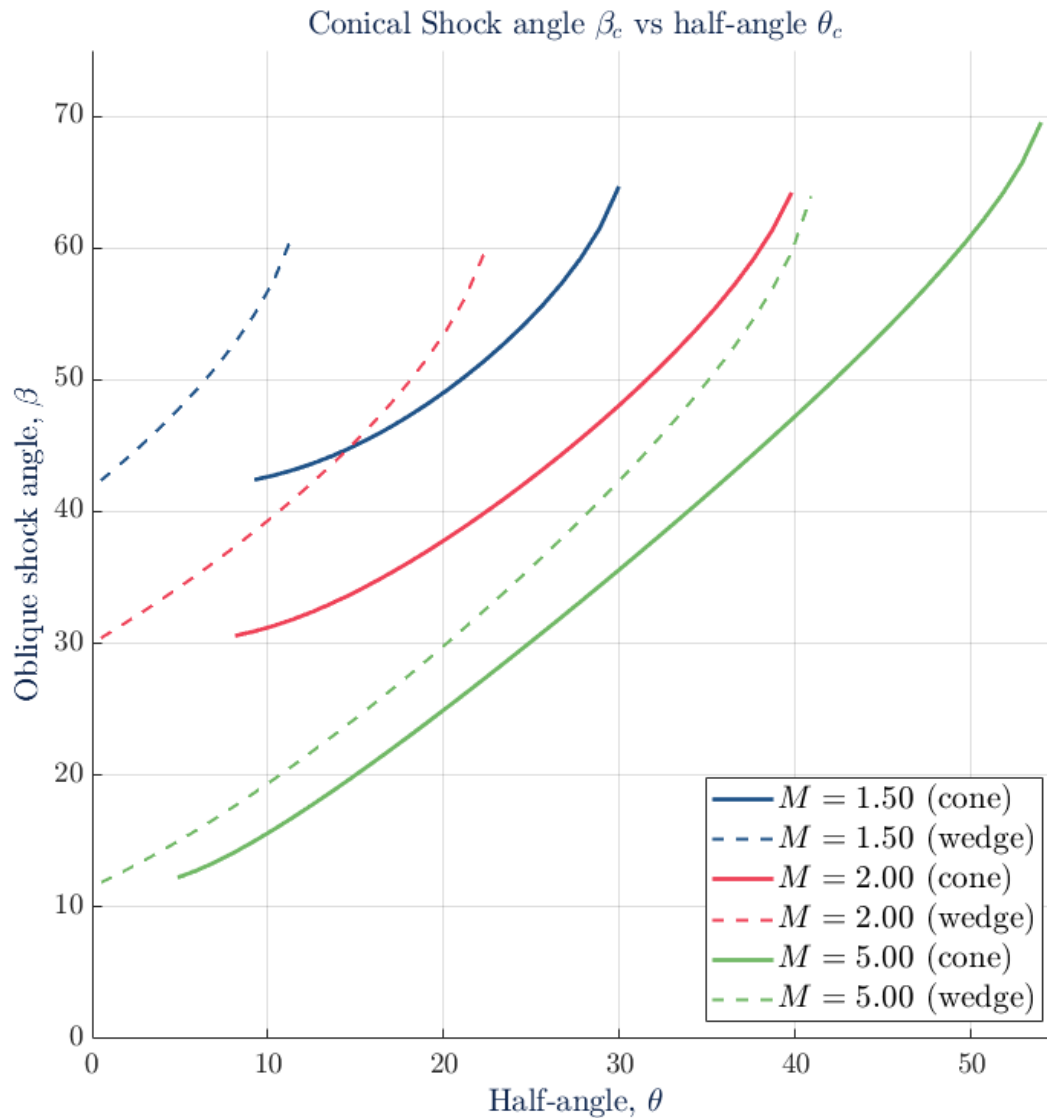
    plot(ax2, theta_c, beta_outputs(i,:), 'LineWidth', lineWidth2,
'Color', color, 'DisplayName', name);
    plot(ax2, theta_c, beta_wedge_outputs(i,:), 'LineWidth',
lineWidth2-0.5, 'Color', color, 'LineStyle', '--', 'DisplayName', name2);
end

% Style
grid(ax2, 'on');
legend(ax2, 'FontSize', legendFontSize2, 'Location', 'southeast');
xlabel(ax2, "Half-angle,  $\theta$ ", 'FontSize', axisFontSize2, 'Color',
axistextcolor);
ylabel(ax2, "Oblique shock angle,  $\beta$ ", 'FontSize', axisFontSize2,
'Color', axistextcolor);
title(ax2, "Conical Shock angle  $\beta_c$  vs half-angle  $\theta$ ",
'Color', titlecolor);

set(ax2, 'FontSize', 14);
ylim(ax2, [0, 75]);
xlim(ax2, [0, 55]);
ax2.XAxisLocation = 'origin';
ax2.YAxisLocation = 'origin';

% 2. Export using 'print' instead of 'exportgraphics'
% This respects the PaperSize and PaperPosition settings exactly.
print(fig2, 'fullpage-plot.pdf', '-dpdf', '-bestfit');

hold(ax2, 'off');
end
```



Solving the Taylor-Maccoll Equations

find the output conical shock angle β given an input M , θ (A.K.A δ), and γ .

1. Assume is a wedge and get M_2 and shock angle β (have functions for them)
2. Find V immediately after shock angle and break into r and θ components
3. Want to find $V_{\theta} = 0$ for BC

```
function outputbeta = coneBeta(Mach, theta, gamma)
% 1. Setup Inputs
M1 = Mach;
theta_cone_input = theta; % degrees
```

```
% 2. Initial Guess
% Get an initial guess for the shock angle beta (assuming it's a wedge)
% This provides a starting point for fzero

% 3. Shooting Method
% fzero tries different beta values until the error is zero
options = optimset('TolX', 1e-4); % Loosen the tolerance for performance

beta_vals = linspace(asind(1/M1)+0.5, 75, 20);
errors = zeros(size(beta_vals));

for k = 1:length(beta_vals)
    errors(k) = solve_for_theta(beta_vals(k), M1, gamma,
theta_cone_input);
end

% Find sign change
idx = find(errors(1:end-1).*errors(2:end) < 0, 1);

if isempty(idx)
    outputbeta = NaN; % no solution
    return;
end

beta_low = beta_vals(idx);
beta_high = beta_vals(idx+1);

outputbeta = fzero(@(b) solve_for_theta(b, M1, gamma, theta_cone_input),
[beta_low beta_high]);

end % End of main function
```

Taylor-Maccoll HELPER FUNCTIONS

Various functions used as part of the process of finding the cone shock angle.

Find corresponding wedge θ for a given conical θ

Inputs:

- Guess of conical oblique shock angle 'beta_guess' (degrees)
- Unperturbed Mach (M1)
- Ratio of specific heats, 'gamma' of flow, usually 1.4
- Given conical half-angle 'theta_target' (degrees)

Outputs:

- Difference between the corresponding θ from the guessed β , and the target θ given. (=0 when the β guess is the correct conical oblique shock)

```
function error = solve_for_theta(beta_guess, M1, gamma, theta_target)
```

```
theta_rad = deg2rad(theta_target);
delta = deltaFinder(M1, beta_guess, gamma);

beta_rad = deg2rad(beta_guess);
delta_rad = deg2rad(delta);

M2 = obliqueMach(M1, beta_guess, delta, gamma);

V_prime = ((2 / ((gamma - 1) * M2^2)) + 1)^-0.5;

vr0 = V_prime * cos(beta_rad - delta_rad);
vt0 = -V_prime * sin(beta_rad - delta_rad); % NOTE: Vt should be
negative here

y0 = [vr0; vt0];
tspan = [beta_rad, deg2rad(2)];

ode45options = odeset('RelTol', 1e-3, 'AbsTol', 1e-5);
[theta_out, y_out] = ode45(@(t, y) taylor_maccoll(t, y, gamma), tspan,
y0, ode45options);

Vt_vals = y_out(:, 2);

% Find sign change with linear interpolation
idx = find(Vt_vals(1:end-1) .* Vt_vals(2:end) < 0, 1);

if isempty(idx)
    % No crossing found: return a large signed error based on
    % whether Vt stayed positive or negative the whole time.
    % This preserves sign information so fzero can bracket.
    error = Vt_vals(end) * 1000;
else
    % Linearly interpolate to find precise theta where Vt = 0
    t1 = theta_out(idx);    t2 = theta_out(idx+1);
    v1 = Vt_vals(idx);      v2 = Vt_vals(idx+1);
    theta_found = t1 - v1 * (t2 - t1) / (v2 - v1);

    error = theta_found - theta_rad;
end
end
```

Taylor-Maccoll equation itself

Inputs:

- Cone half-angle 'theta' (degrees)
- Vector of velocity components [radial direction, angular direction] 'y'
- Ratio of specific heats, 'gamma' of flow, usually 1.4

Outputs:

- Numeric derivatives of velocity in the r, θ direction.


```
function dydt = taylorMaccoll(theta, y, gamma)
    Vr = y(1);
    Vt = y(2);

    % The Taylor-Maccoll Equation
    numerator = ((gamma-1)/2)*(1-Vr^2-Vt^2)*(2*Vr+Vt*cot(theta))-(Vr*Vt^2);
    denominator = Vt^2-((gamma-1)/2)*(1-Vr^2-Vt^2);
    if abs(denominator) < 1e-6
        denominator = sign(denominator)*1e-6;
    end

    dVr = Vt;
    dVt = numerator / denominator;

    dydt = [dVr; dVt];

end
```

Generic Helper Functions

Below is a set of functions used, based on lectures, NASA, and other sources. Mainly just given flow relations for known cases.

Oblique shock resultant Mach

Inputs:

- Unperturbed flow mach number 'M' (often notated as 'M1' in diagrams)
- Oblique shock angle 'beta' (in degrees)
- Wall/flow deflection angle 'delta' (in degrees)
- Flow ratio of specific heats 'gamma,' usually 1.4

Outputs:

- Mach number after oblique shock & deflection, often notated as 'M2'

```
function mach = obliqueMach(M, beta, delta, gamma)
    mach = (((gamma-1)*(M^2)*((sind(beta))^2)+2) / (2*gamma*(M^2)*
    ((sind(beta))^2) - (gamma-1)))^0.5 / (sind(beta-delta));
end
```

Find oblique shock angle "beta" instead of using the table.

Inputs:

- Unperturbed mach "M" (AKA 'M1' in most notation),
- wall/flow deflection angle "theta" (often notated as "delta"),
- ratio of specific heats "gamma" of the flow,
- "n" specifies weak (n=0) or strong (n=1) shock case. (Usually should be weak shock case.)

Outputs:

- Oblique shock angle "beta" in degrees.

```
function Beta=beta(M,theta,gamma,n)
    theta=theta*pi/180;           % convert to radians
    mu=asin(1/M);                 % Mach wave angle
    c=tan(mu)^2;
    a=(gamma-1)/2+(gamma+1)*c/2*tan(theta);
    b=(gamma+1)/2+(gamma+3)*c/2*tan(theta);
    d=sqrt(4*(1-3*a*b)^3/((27*a^2*c+9*a*b-2)^2)-1);
    Beta=atan((b+9*a*c)/(2*(1-3*a*b))-(d*(27*a^2*c+9*a*b-2))/(
    (6*a*(1-3*a*b))*tan(n*pi/3+1/3*atan(1/d))))*180/pi;

    %{
    Source:
    Based on an analytical solution to the theta-beta-Mach relation given in
    the following reference:  Rudd, L., and Lewis, M. J., "Comparison of
    Shock Calculation Methods", AIAA Journal of Aircraft, Vol. 35, No. 4,
    July-August, 1998, pp. 647-649.

    Original script by Chris Plumley, undergraduate, University of Maryland.
    %}
end
```

Find max wedge shock angle for a given mach

Inputs:

- Unperturbed mach "M" (AKA 'M1' in most notation),
- ratio of specific heats "gamma" of the flow,

Outputs:

- Max weak shock angle "beta" in degrees.
- Half-angle "theta" for this max oblique shock angle, in degrees.

```
function [maxTheta, maxBeta] = maxWedgeBeta(M, gamma)

t = @(b) deltaFinder(M, b, gamma);
eps = 1e-7;

dtDb = @(b) (t(b+eps) - t(b-eps))/(2*eps);

% Use a guess for Beta
% 65 degrees is a safe starting point for most supersonic Mach numbers.
maxBeta = fzero(dtDb, 65); % guess 65 degrees at first
maxTheta = deltaFinder(M, maxBeta, gamma);

end
```

Find delta when given M and beta (degrees)

Inputs:

- Unperturbed Mach (AKA M_1)
- oblique shock angle β (degrees)
- Ratio of specific heats, ' γ ' of flow, usually 1.4

Outputs:

- Wall deflection angle ' δ ' (in degrees)

```
function delta = deltaFinder(M, beta, gamma)
    delta = ( ( M^2 * (sind(beta)^2) ) - 1) / ( M^2 * (gamma + cosd(2*beta))
+ 2 );
    delta = 2*cotd(beta)*delta;
    delta = atand(delta);
end
```

Published with MATLAB® R2024b